

FastXeBench: A Reproducible Benchmark and Fidelity Study of Edit-to-PDF Latency for XeLaTeX on Windows

Zirui Xu
Nanjing Normal University
19230444@njnu.edu.cn

Abstract

FastXeBench is a reproducible benchmark study of edit-to-PDF latency for XeLaTeX on Windows. The harness uses a prime-plus-measure protocol so that incremental scenarios time only the post-edit build rather than a fresh build. Our formal core matrix covers four workloads, five scenarios, and three main configurations, producing 45 workload-scenario-config groups and 900 recorded runs. We report per-group medians, IQRs, and bootstrap 95% confidence intervals, with corrected nonparametric comparisons deferred to the appendix. The results are workload-sensitive: **latexmk** reliably improves steady incremental edits, but it is not universally faster on clean builds or bibliography edits, while **minted** cache is the strongest safe optimization on code-heavy documents. All main-matrix runs preserve visual equality with the baseline. Two real-project-derived validation workloads reproduce the **latexmk** pattern, and a scoped Windows storage follow-up shows that the observed C: profile root is about 1.20x slower than the repository-local D: root. These results position FastXeBench as both a benchmark artifact and a practical guide for Windows-local XeLaTeX workflows.

1 Introduction

XeLaTeX users often optimize their local edit-compile loop without a reproducible method for comparing toolchain choices. Recent evidence that TeX outputs can vary across engines and distribution versions means that latency alone is not a sufficient optimization target; fidelity must be measured alongside speed [9]. FastXeBench targets that gap with a runnable benchmark harness centered on edit-to-PDF latency rather than fresh-build latency.

This paper makes four concrete contributions. First, it packages a Windows-local XeLaTeX benchmark artifact with explicit workload, scenario, and configuration boundaries. Second, it fixes the measurement target at edit-to-PDF latency through a prime-plus-measure protocol rather than a fresh-build proxy. Third, it treats output fidelity as a release gate by combining PDF hashes with page-level raster comparison. Fourth, it turns the resulting dataset into practical guidance: **latexmk** is a steady incremental default, **minted** cache is the strongest safe code-heavy optimization in the present matrix, and excluded configurations are kept explicit rather than folded into optimistic recommendations.

2 Background and Positioning

FastXeBench is positioned as a Windows-local benchmark and engineering study rather than a new compiler. The comparison space is grounded in existing TeX tooling: **latexmk** automates reruns and dependency tracking [4], **fontspec** exposes the native-font path that makes XeLaTeX attractive on Windows [3], **minted** adds a realistic cache-heavy code-highlighting workload [5], **mylatexformat** represents preamble dumping [6], PGF/TikZ covers figure-heavy workflows [7], and Tectonic provides a modern cached-engine comparison point [10]. The paper is also

Table 1: Experimental setup for the formal core dataset.

Item	Value
Host CPU	12th Gen Intel(R) Core(TM) i7-12700H (14 cores / 20 threads)
Memory	15.6 GiB physical RAM
OS	Microsoft Windows 11 Home (Chinese edition), version 10.0.26100 (build 26100)
Machine	ASUS TUF Gaming F15 FX507ZV4_FX507ZV4
Power plan	Balanced
Repository volume	D: NTFS fixed volume; Defender performance mode status 0
Primary TeX engine	XeTeX 3.141592653-2.6-0.999997 (TeX Live 2025)
Automation	Latexmk, John Collins, 15 June 2025. Version 4.87; hyperfine 1.20.0
Follow-up engine	teXtonic 0.15.0
PDF comparison	pdftoppm version 25.02.0 at 144 dpi plus per-page PNG SHA-256
Font path	<code>ctexart</code> on the local Windows CJK path; R1 validation uses TeX Gyre Pagella, Heros, and Cursor.
Formal protocol	<code>v0.2-prime-measure</code> ; 4 workloads, 5 scenarios, 3 main configurations
Runs per group	3 warmups and 20 recorded runs for the formal core matrix

prepared with an arXiv-first submission path in mind, so current arXiv XeLaTeX and TeX Live constraints matter directly [2, 1].

3 Benchmark Design

Protocol v0.2 uses prime-plus-measure semantics for incremental scenarios: a stable state is built in an isolated run directory, a controlled edit is applied, and only the second build is timed. The current formal core matrix covers four workloads, five scenarios, and three main-matrix configurations (B0, B1, and B4). The workload set intentionally spans text-and-math, CJK/native-font, TikZ, and code-heavy documents, with the native-font and code-heavy paths explicitly exercising `fontspec` and `minted` behavior [3, 5]. Synthetic workloads provide controlled comparisons, while second-batch follow-up datasets add two real-project-derived validations and one scoped Windows storage comparison.

All formal measurements use the single Windows-local host summarized in Table 1. Each recorded run executes in an isolated directory under the dataset root. Scenario `S0` measures a clean build directly, while `S1`–`S4` first build a stable state, apply one controlled mutation, and then time only the post-edit rebuild. Reference PDFs are generated per workload/scenario pair and reused for fidelity comparisons.

Fidelity is checked in two stages. The harness first records a whole-file SHA-256 for the candidate PDF and its workload/scenario reference. It then rasterizes both PDFs with `pdftoppm -png -r 144` and compares per-page PNG hashes. If any page hash differs, the run is marked `visual_equal = false` and the changed page indices are recorded together with diagnostic diff images. In the present paper, `binary_equal` is retained as a secondary indicator only; the main fidelity gate is `visual_equal`.

Primary statistical summaries use group medians, IQRs, and 95% bootstrap confidence intervals computed from the recorded wall-clock latencies. Appendix Table 3 adds Holm-corrected two-sided Wilcoxon signed-rank tests for the non-baseline configurations. Because the current artifact records repeated runs rather than an explicit cross-configuration pairing ID, these appendix tests pair runs by within-group recorded order after sorting `run_id`; we therefore treat them as supporting evidence rather than stronger than the descriptive summaries themselves.

Table 2: Formal core rollup for non-baseline configurations. Incremental latency is the median over S1, S3, and S4.

Workload	Config	Clean (s)	Clean $\times$	Bib (s)	Bib $\times$	Incremental (s)	Incremental $\times$
W1 Text+Math	B1 latexmk	8.02	0.88	8.02	0.88	4.02	1.76
W2 CJK+Font	B1 latexmk	9.02	0.78	9.02	0.78	4.02	1.76
W4 TikZ	B1 latexmk	10.02	1.01	10.02	1.01	4.02	2.51
W5 Minted	B1 latexmk	38.02	1.13	38.01	1.13	14.01	3.07
W5 Minted	B4 minted cache	19.02	2.26	16.02	2.69	6.01	7.16

4 Acceleration Techniques

The current harness includes a direct XeLaTeX baseline, `latexmk` automation [4], `minted` cache reuse [5], a negative XeLaTeX-plus-`mylatexformat` compatibility case [6], TikZ externalization as an appendix-only negative result rooted in PGF/TikZ workflows [7], and a Tectonic comparison that is retained as a development-only reference because of persistent fidelity differences [10].

5 Results

The formal core dataset contains 45 workload-scenario-config groups and 900 recorded runs. All recorded runs succeeded, and every main-matrix row currently preserves visual equality with the baseline. This means the primary conclusions are driven by steady-state latency rather than speed-versus-fidelity trade-offs inside the core matrix. Table 2 compresses the main matrix, while Appendix Table 3 reports the per-group medians, IQRs, bootstrap confidence intervals, and corrected nonparametric tests.

Figure 1 shows that the main result is workload-sensitive rather than uniform. For the W1 text+math and W2 CJK+font workloads, `latexmk` reduces the S1, S3, and S4 edit latencies from about 7.1 seconds to about 4.0 seconds, yielding roughly 1.76x speedups, but it is slower than the baseline on clean builds and bibliography edits. For W4 TikZ, `latexmk` is close to neutral on clean builds and bibliography edits, while reducing text, figure, and preamble edits from about 10.1 seconds to about 4.0 seconds, for about 2.50x speedups. For W5 minted, `latexmk` lowers clean builds from about 43.1 seconds to about 38.0 seconds, and lowers S1, S3, and S4 to about 14.0 seconds, for about 3.07x speedups.

`minted` cache is the strongest stable optimization in the current core matrix. On W5 minted, it reduces clean-build latency to about 19.0 seconds (2.26x over baseline), reduces bibliography edits to about 16.0 seconds (2.69x), and reduces text, figure, and preamble edits to about 6.0 seconds (about 7.16x), while preserving visual equality in all recorded runs. Figure 2 makes this separation visible at the run level rather than only at the level of group medians.

Appendix Table 3 shows that these group differences remain statistically distinguishable after Holm correction. That result should be read carefully: corrected significance does not imply universal speedup, because several B1 clean-build and bibliography cells are significantly slower than the direct XeLaTeX baseline even while the incremental cells remain significantly faster.

The second-batch follow-up results support the same interpretation. In the external validation dataset, two real-project-derived workloads, R1 (a `fontspec`-oriented example) and R2 (a `pgfplots`-oriented example), reproduce the `latexmk` pattern seen in the synthetic matrix: clean builds and bibliography edits are slower than the direct XeLaTeX baseline at about 0.78x baseline speed, while incremental edits still improve by about 1.76x, again with full visual equality. In the B7 storage dataset, the passive Windows storage comparison between `d_repo_ntfs` and `c_profile_ntfs` preserves visual equality in all nine groups, but the `c_profile_ntfs` root is

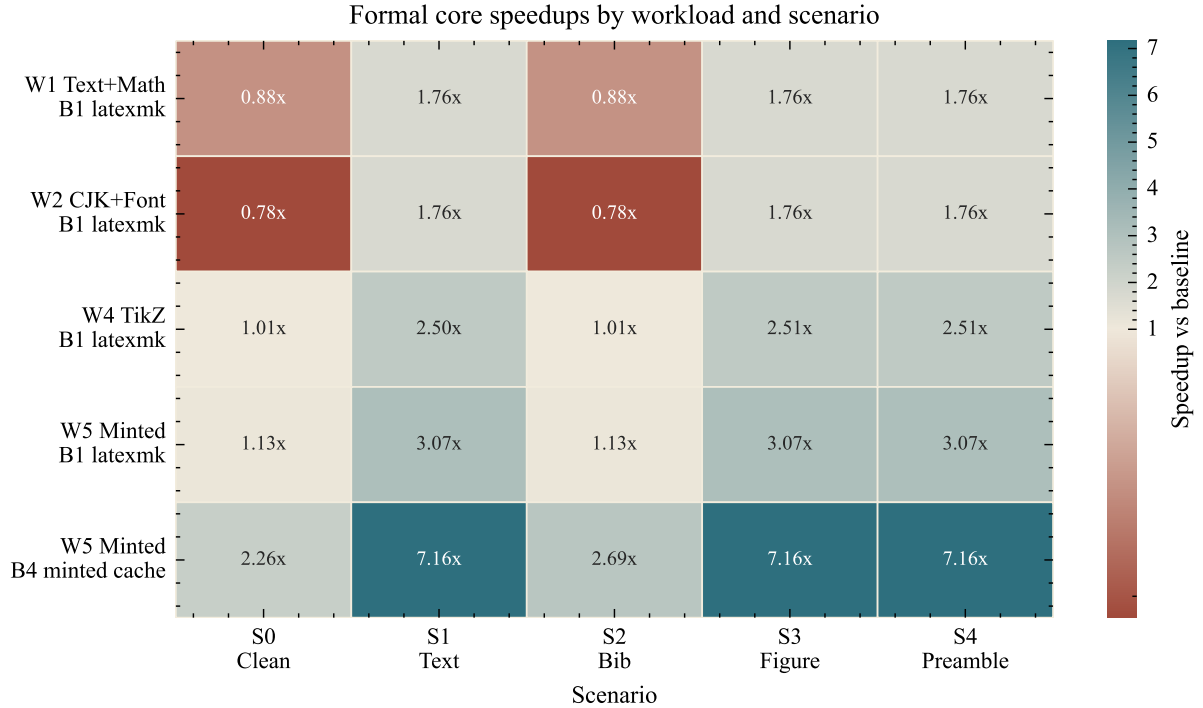


Figure 1: Formal speedup matrix for the non-baseline configurations. Values above 1.0 indicate lower latency than the XeLaTeX baseline.

slower overall, with a median latency ratio of about 1.20x relative to the repository-local D: root. Appendix Tables 4 and 5 summarize these follow-up datasets directly.

6 Threats to Validity

The current artifact still runs on a single Windows host, so the reported latency distributions inevitably include host-specific effects such as storage layout, antivirus state, and local font availability. The workload mix also remains partly synthetic even after the two real-project-derived validation workloads were added, which means the benchmark is stronger as a controlled comparative study than as a claim about every class of TeX project. The storage follow-up is especially scoped: Dev Drive is intended as a dedicated Windows developer volume [8], but the current environment does not permit a full privileged Dev Drive provisioning or direct performance-mode manipulation. The benchmark therefore reports a Windows-local storage-root comparison under the observed system state, not a universal claim about all Dev Drive configurations. Finally, the current paper emphasizes descriptive results and reproducible artifacts; the appendix-level Wilcoxon analysis is informative, but it depends on recorded-order pairing rather than an explicit synchronized cross-config run identifier.

7 Conclusion

The current evidence supports a conservative recommendation set. For steady incremental writing loops, `latexmk` is the default win, but its gains are workload- and scenario-dependent rather than universal. For code-heavy documents, `minted` cache is the strongest safe optimization in the present matrix. Fidelity should remain a release gate, not an afterthought, because configuration choices that look attractive from latency alone can still produce visible output differences. This leaves FastXeBench in a useful position for an arXiv-first release: the benchmark results

Representative latency distributions from the formal core dataset

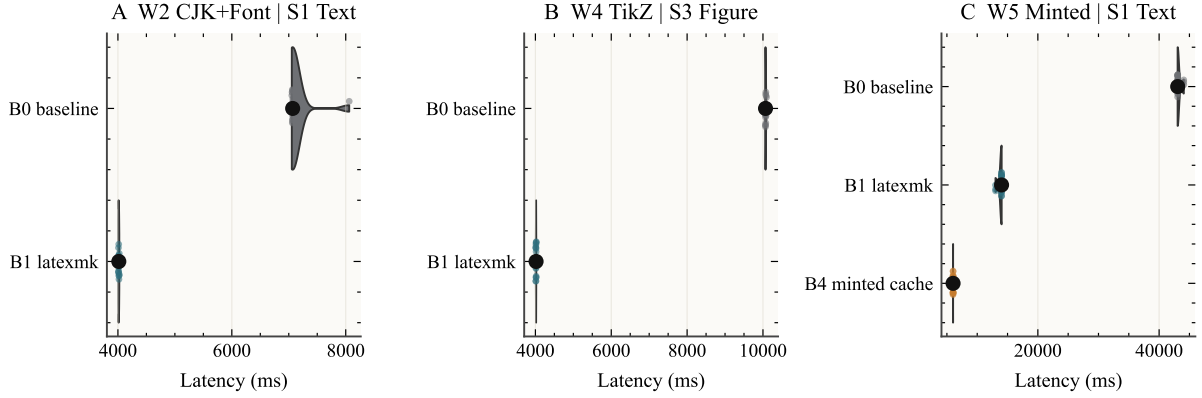


Figure 2: Representative per-run latency distributions. Panel A shows W2 text edits, panel B shows W4 figure edits, and panel C shows W5 text edits with both `latexmk` and `minted` cache.

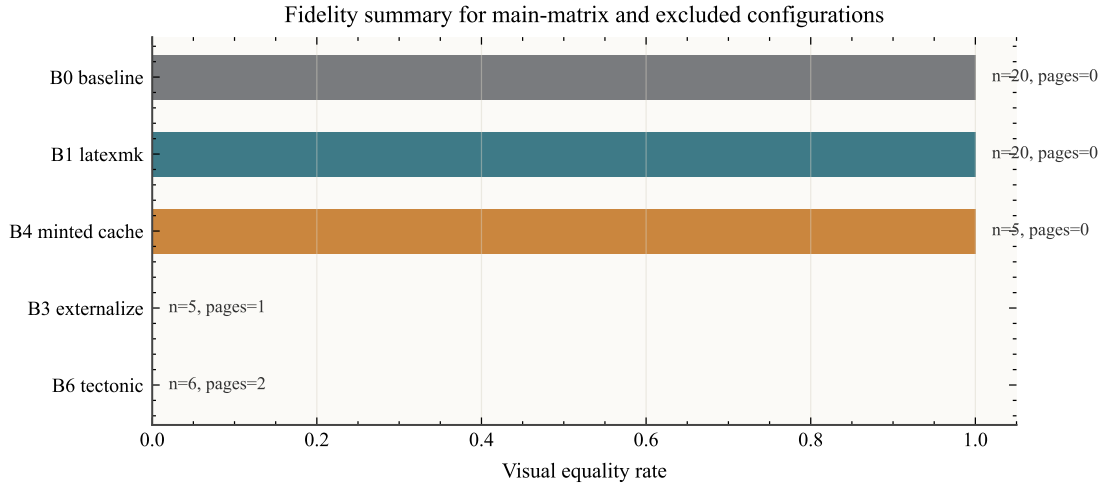


Figure 3: Visual-equality summary for the main-matrix configurations and the excluded B3/B6 diagnostic paths. The core matrix remains fully visually equal, while triage reproduces persistent fidelity failures for the excluded configurations.

are stable, the excluded configurations are explicitly scoped, and the repository now supports both reruns and paper regeneration from the recorded datasets.

8 Artifact Availability

The benchmark workloads, orchestration scripts, clean dataset layout, and environment meta-data are publicly available at <https://github.com/handsomeZR-netizen/fastxebench>. The repository is intended to expose the benchmark harness, paper sources, analysis scripts, and compact result artifacts needed to regenerate the figures and tables, while larger raw datasets may be distributed as release assets. The current submission path is intentionally arXiv-first. arXiv currently supports TeX Live 2023 and 2025, with 2025 as the default, and provides specific XeLaTeX guidance for filename-based `fontspec` loading and `minted-v3` cache generation [2]. If a multilingual version is prepared later, the English version should appear first [1]. To reduce packaging risk, the repository also includes a local dry-run checklist and isolated submission-bundle script.

A Appendix Statistics

B Excluded Configurations and Scoped Follow-up

B2 remains a compatibility-limited XeLaTeX negative result tied to the `mylatexformat` path [6]. B3 delivered speedups on TikZ-heavy cases but produced persistent visual differences, so it is retained as an appendix-only negative result rather than a recommendation. B6 remains development-only: Tectonic provides a useful cached-engine comparison point [10], but its output differences were stable enough in triage to keep it out of the main matrix. B7 should be read as a scoped Windows storage-root comparison rather than a complete Dev Drive experiment, because the current environment can observe Microsoft Defender and volume metadata but cannot exercise the full privileged Dev Drive path [8].

References

- [1] arXiv. Non-english submissions. <https://info.arxiv.org/help/faq/multilang.html>, 2026. Accessed 2026-03-21.
- [2] arXiv. TeX Live at arxiv. <https://info.arxiv.org/help/faq/texlive.html>, 2026. Accessed 2026-03-21.
- [3] CTAN Team. fontspec – advanced font selection in XeLaTeX and LuaLaTeX. <https://ctan.org/pkg/fontspec>, 2026. Accessed 2026-03-21.
- [4] CTAN Team. latexmk – fully automated LaTeX document generation. <https://ctan.org/pkg/latexmk>, 2026. Accessed 2026-03-21.
- [5] CTAN Team. minted – highlighted source code for LaTeX. <https://ctan.org/pkg/minted>, 2026. Accessed 2026-03-21.
- [6] CTAN Team. mylatexformat – precompile a LaTeX document into a format. <https://ctan.org/pkg/mylatexformat>, 2026. Accessed 2026-03-21.
- [7] CTAN Team. pgf – create PostScript and PDF graphics in TeX. <https://ctan.org/pkg/pgf>, 2026. Accessed 2026-03-21.
- [8] Microsoft. Set up a dev drive on Windows 11. <https://learn.microsoft.com/en-us/windows/dev-drive/>, 2026. Microsoft Learn, accessed 2026-03-21.
- [9] Jovyn Tan and Manuel Rigger. Inconsistencies in tex-produced documents. *arXiv preprint arXiv:2407.15511*, 2024.
- [10] Tectonic Project. Building documents with the Tectonic command-line tool. <https://tectonic-typesetting.github.io/book/latest/v2cli/build.html>, 2026. Accessed 2026-03-21.

Table 3: Appendix statistics for the non-baseline main-matrix configurations. IQR is reported as $[Q_1, Q_3]$, CI is the 95% bootstrap interval for the group median, and Holm-adjusted p values come from two-sided Wilcoxon signed-rank tests that pair runs by within-group recorded order after sorting by `run_id`.

Workload	Scenario	Config	Median (s)	IQR (s)	95% CI (s)	Speedup	Holm p
W1 Text+Math	S0 clean	B1 latexmk	8.02	[8.01, 8.02]	[8.01, 8.02]	0.88	<0.001
W1 Text+Math	S1 text	B1 latexmk	4.02	[4.01, 4.02]	[4.01, 4.02]	1.76	<0.001
W1 Text+Math	S2 bib	B1 latexmk	8.02	[8.01, 8.02]	[8.01, 8.02]	0.88	<0.001
W1 Text+Math	S3 figure	B1 latexmk	4.02	[4.01, 4.02]	[4.01, 4.02]	1.76	<0.001
W1 Text+Math	S4 preamble	B1 latexmk	4.02	[4.01, 4.02]	[4.01, 4.02]	1.76	<0.001
W2 CJK+Font	S0 clean	B1 latexmk	9.02	[9.01, 9.02]	[9.01, 9.02]	0.78	<0.001
W2 CJK+Font	S1 text	B1 latexmk	4.02	[4.01, 4.02]	[4.01, 4.02]	1.76	<0.001
W2 CJK+Font	S2 bib	B1 latexmk	9.02	[9.01, 9.02]	[9.01, 9.02]	0.78	<0.001
W2 CJK+Font	S3 figure	B1 latexmk	4.01	[4.01, 4.02]	[4.01, 4.02]	1.76	<0.001
W2 CJK+Font	S4 preamble	B1 latexmk	4.02	[4.01, 4.02]	[4.01, 4.02]	1.76	<0.001
W4 TikZ	S0 clean	B1 latexmk	10.02	[10.01, 10.02]	[10.01, 10.02]	1.01	<0.001
W4 TikZ	S1 text	B1 latexmk	4.02	[4.02, 4.02]	[4.02, 4.02]	2.50	<0.001
W4 TikZ	S2 bib	B1 latexmk	10.02	[10.01, 10.02]	[10.02, 10.02]	1.01	<0.001
W4 TikZ	S3 figure	B1 latexmk	4.02	[4.01, 4.02]	[4.02, 4.02]	2.51	<0.001
W4 TikZ	S4 preamble	B1 latexmk	4.01	[4.01, 4.02]	[4.01, 4.02]	2.51	<0.001
W5 Minted	S0 clean	B1 latexmk	38.02	[38.01, 38.02]	[38.01, 38.02]	1.13	<0.001
W5 Minted	S0 clean	B4 minted cache	19.02	[19.01, 19.02]	[19.01, 19.02]	2.26	<0.001
W5 Minted	S1 text	B1 latexmk	14.02	[14.01, 14.02]	[14.01, 14.02]	3.07	<0.001
W5 Minted	S1 text	B4 minted cache	6.02	[6.01, 6.02]	[6.01, 6.02]	7.16	<0.001
W5 Minted	S2 bib	B1 latexmk	38.01	[38.01, 38.02]	[38.01, 38.02]	1.13	<0.001
W5 Minted	S2 bib	B4 minted cache	16.02	[16.01, 16.02]	[16.01, 16.02]	2.69	<0.001
W5 Minted	S3 figure	B1 latexmk	14.01	[13.76, 14.01]	[14.01, 14.01]	3.07	<0.001
W5 Minted	S3 figure	B4 minted cache	6.01	[6.01, 6.02]	[6.01, 6.01]	7.16	<0.001
W5 Minted	S4 preamble	B1 latexmk	14.01	[14.01, 14.02]	[14.01, 14.02]	3.07	<0.001
W5 Minted	S4 preamble	B4 minted cache	6.01	[6.01, 6.02]	[6.01, 6.02]	7.16	<0.001

Table 4: External validation rollup for the two real-project-derived workloads.

Workload	Clean $\times$	Bib $\times$	Incremental $\times$	Visual eq.
R1 fontspec	0.78	0.78	1.76	1.00
R2 pgfplots	0.79	0.78	1.76	1.00

Table 5: Scoped Windows storage follow-up. The baseline mode is the repository-local D: root.

Mode	Groups	Volume	Median ratio	Visual eq.
d_repo_ntfs	9	D:/NTFS	1.00	1.00
c_profile_ntfs	9	C:/NTFS	1.20	1.00